

Паттерны работы с каналами

Игорь Шамаев
Руководитель направления разработки

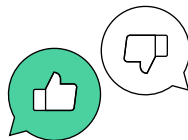


Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

-  если меня видно и слышно
-  если нет

Рекомендации

- Если смотрите с компьютера
 - Используйте браузеры **Google Chrome** или **Microsoft Edge**
 - Если есть проблемы с изображением или звуком, обновите страницу — **F5**
- Если смотрите с мобильного телефона или планшета
 - Перейдите с мобильного интернет-соединения на **Wi-Fi**
 - Если есть проблемы с изображением или звуком, перезапустите приложение **МТС Линк** на телефоне
 - Предварительно проверьте, подходит ли ваше устройство для подключения к вебинару, по [ссылке](#)

Правила участия

- 1 Приготовьте блокнот и ручку, чтобы записывать важные мысли и идеи
- 2 Продолжительность вебинара — 1 час
- 3 Вы можете писать свои вопросы в чате
- 4 Запись вебинара будет доступна в личном кабинете



Шамаев Игорь

О спикере:

- Руководитель направления разработки в Домклик
- Развивает и поддерживает внутреннюю базу данных управления конфигурацией на Golang и TypeScript
- В прошлом — руководитель команды разработки в Сбере

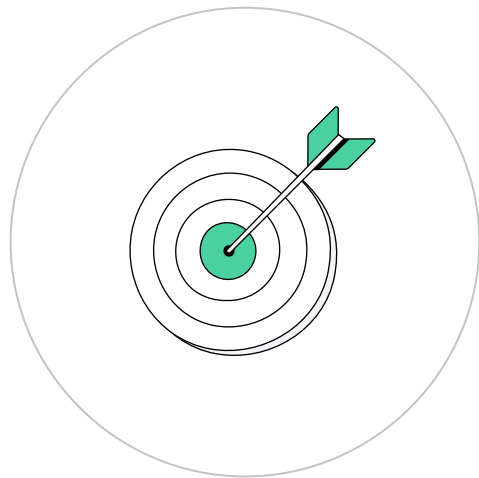




**Пробовали ли вы уже
сами что-то написать
с использованием
каналов?**

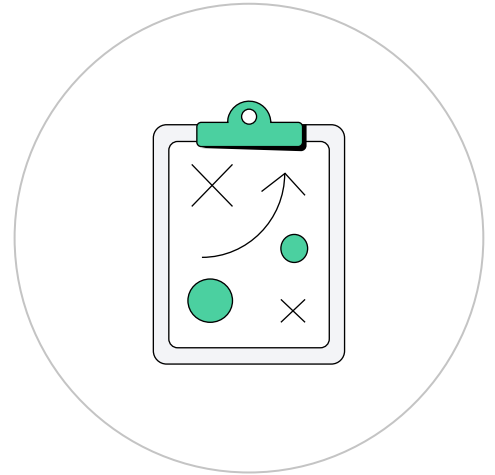
Цели занятия

- Разобрать 3 ключевых паттерна: Fan-out, Fan-in и Worker Pool
- Научиться применять паттерны в реальных задачах
- Понимать плюсы/минусы каждого подхода
- Избегать типичных ошибок и deadlock'ов



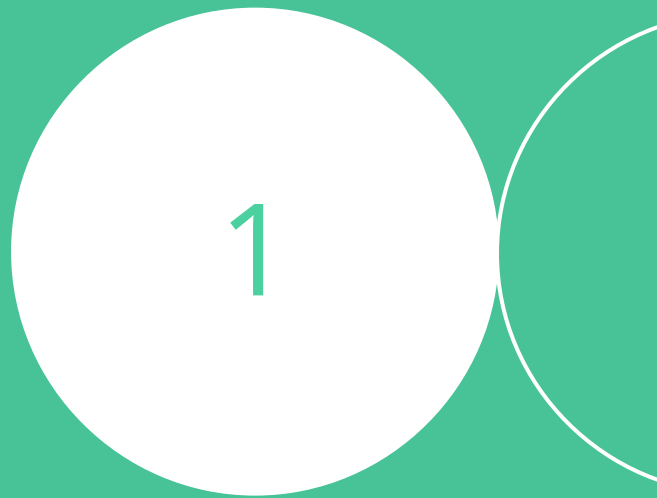
План занятия

- 1 Fan-out: распределение задач
- 2 Fan-in: объединение потоков
- 3 Worker Pool: управление ресурсами
- 4 Сравнение паттернов
- 5 Практика



Fan-out

Распределение задач



Что такое Fan-out

- Паттерн для распределения задач между несколькими обработчиками
- Один входной канал → несколько горутин-обработчиков
- Увеличивает скорость обработки за счёт параллелизма

```
input := make(chan int)
for i := 0; i < 5; i++ {
    go worker(input) // 5 обработчиков
}
```

Схема Fan-out

[Источник данных]

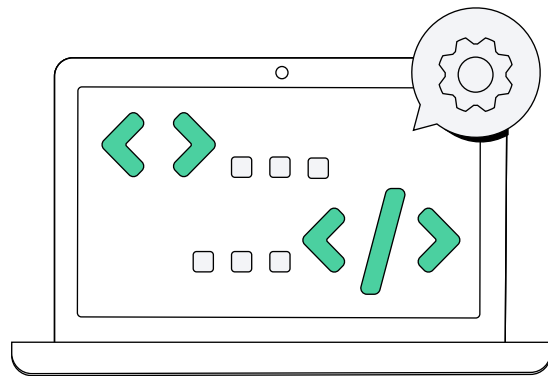


[Канал]



/---|---|\

[W1][W2][W3] (Горутины-обработчики)



Когда использовать Fan-out

- Обработка независимых элементов данных
- Вычислительно дорогие операции
- Параллельная обработка запросов
- Чтение из медленных источников (файлы, сеть)

Базовый пример Fan-out

```
func main() {
    jobs := make(chan int, 10)

    // Запуск 3 воркеров
    for w := 1; w <= 3; w++ {
        go worker(w, jobs)
    }

    // Отправка задач
    for j := 1; j <= 5; j++ {
        jobs <- j
    }

    close(jobs)
}

func worker(id int, jobs <-chan int) {
    for j := range jobs {
        fmt.Printf("Worker %d processing job %d\n", id, j)
        time.Sleep(time.Second) // Имитация работы
    }
}
```

Преимущества Fan-out

- Простота реализации
- Максимальное использование CPU
- Увеличение пропускной способности
- Контроль за потреблением памяти

Риски Fan-out

- Неконтролируемое создание горутин
- Потенциальные утечки памяти
- Отсутствие порядка результатов
- Сложность обработки ошибок

Как ограничить количество горутин

- Использовать семафоры через буферизованные каналы
- Использовать пул воркеров
- Реализовать механизм graceful shutdown

```
var sem = make(chan struct{}, 10) // Ограничение 10 горутин

func process(data Data) {
    sem <- struct{}{}           // Захват семафора
    defer func() { <-sem }()    // Освобождение

    // Обработка данных
}
```


Обработка ошибок в Fan-out

- Создать канал для ошибок
- Использовать `select` для мониторинга
- Реализовать механизм отмены через `context`

```
errCh := make(chan error)
ctx, cancel := context.WithCancel(context.Background())
defer cancel()
```

Контроль времени выполнения

→ Использование context.WithTimeout

→ Таймеры в select

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

go func() {
    select {
    case <-ctx.Done():
        fmt.Println("Timeout exceeded")
        return
    case result := <-dataCh:
        // Обработка результата
    }
}()
```

Вопрос

Как вы думаете, в каких случаях паттерн Fan-out может привести к избыточному потреблению ресурсов?



Ответ

- Количество горутин не ограничено
- Обработчики потребляют много памяти
- Задачи неравномерные по времени выполнения
- Источник данных быстрее, чем обработчики

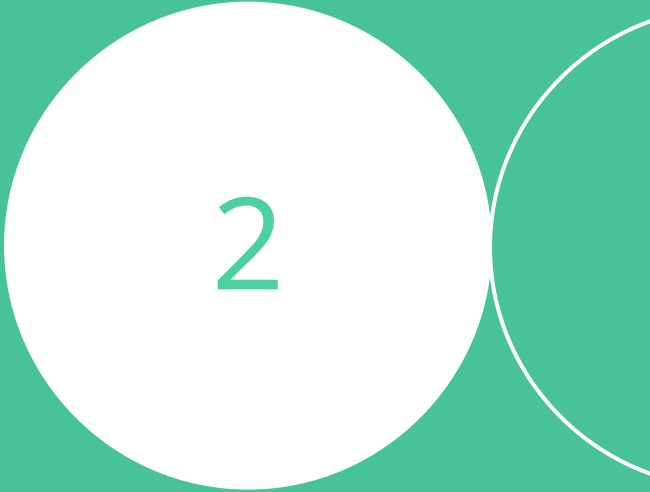




Ваши вопросы?

Fan-in

Объединение потоков



2

The diagram illustrates a Fan-in operation. It features two overlapping circles on the right side of the slide. The left circle is white with a green number '2' in the center. The right circle is partially visible, showing a green outline and a teal interior. The background is a solid teal color.

Что такое Fan-in

- Паттерн для объединения данных из нескольких источников
- Множество входных каналов → один выходной канал
- Позволяет консолидировать результаты

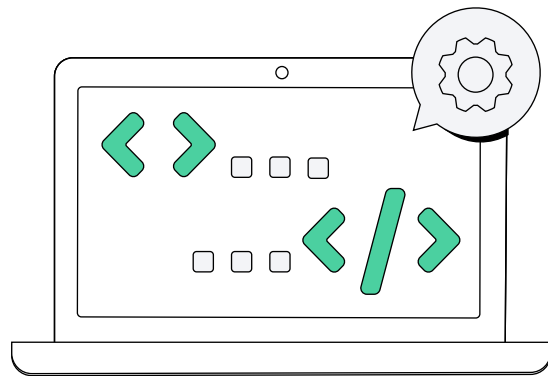
```
func merge(cs ...<-chan int) <-chan int {  
    out := make(chan int)  
    // ... логика объединения ...  
    return out  
}
```

Схема Fan-in

[Источник 1] → [Канал 1] \

[Источник 2] → [Канал 2] ----→ [Объединённый канал]

[Источник 3] → [Канал 3] /



Когда использовать Fan-in

- Агрегация данных из нескольких источников
- Объединение результатов параллельных вычислений
- Сбор метрик/логов из разных компонентов
- Реализация publish-subscribe систем

Базовый пример Fan-in

```
func merge(channels ...<-chan int) <-chan int {
    var wg sync.WaitGroup
    out := make(chan int)

    // Запуск обработчика для каждого канала
    for _, c := range channels {
        wg.Add(1)
        go func(c <-chan int) {
            defer wg.Done()
            for n := range c {
                out <- n
            }
        }(c)
    }

    // Закрытие выходного канала после завершения
    go func() {
        wg.Wait()
        close(out)
    }()

    return out
}
```

Преимущества Fan-in

- Упрощение потребления данных
- Централизация обработки
- Возможность фильтрации данных
- Гарантия порядка элементов

Риски Fan-in

- Возможность deadlock'ов
- Неправильное закрытие каналов
- Блокировка при блокировке одного источника
- Сложность отладки

Заккрытие каналов в Fan-in

- Все входные каналы должны быть закрыты
- Использовать `sync.WaitGroup` для отслеживания
- Закрывать выходной канал после завершения всех горутин

```
go func() {  
    wg.Wait()  
    close(out) // Безопасное закрытие  
}()
```

Обработка блокировок

- Использование select с таймаутами
- Неблокирующая отправка данных

```
select {  
  case out <- data: // Успешная отправка  
  case <-time.After(100 * time.Millisecond):  
    log.Println("Timeout sending data")  
  case <-ctx.Done():  
    return  
}
```



**Что страшнее: забыть
закрыть канал в Fan-in
или сделать это
дважды?**

Ответ

- Заккрытие закрытого канала → panic
- Незакрyтый канал → возможная утечка горутин

Лучшая практика:

- Закрывать каналы только отправителем
- Использовать sync.Once для безопасного закрытия
- Проверять состояние канала через ok

```
defer func() {  
    if !closed {  
        close(ch)  
        closed = true  
    }  
}()
```




Ваши вопросы?

Worker Pool

Управление ресурсами



3

Что такое Worker Pool

- Паттерн с фиксированным количеством обработчиков
- Очередь задач → пул воркеров
- Контроль за потреблением ресурсов

```
tasks := make(chan Task, 100)
for i := 0; i < 10; i++ {
    go worker(tasks) // 10 воркеров
}
```

Схема Worker Pool

[Генератор задач]

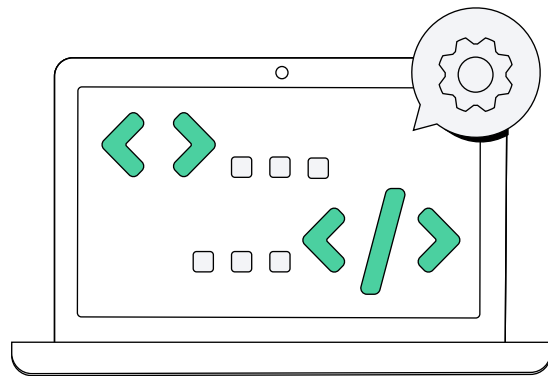


[Очередь задач (канал)]



/---|---|\

[W1][W2][W3] (Фиксированные воркеры)



Когда использовать Worker Pool

- Ограничение параллелизма
- Обработка задач с переменной нагрузкой
- Системы с ограниченными ресурсами
- Балансировка нагрузки
- Очереди обработки запросов

Базовый пример Worker Pool

```
func main() {
    const numWorkers = 5
    tasks := make(chan int, 10)
    var wg sync.WaitGroup

    // Запуск воркеров
    for i := 1; i <= numWorkers; i++ {
        wg.Add(1)
        go worker(i, tasks, &wg)
    }

    // Отправка задач
    for j := 1; j <= 20; j++ {
        tasks <- j
    }

    close(tasks)
    wg.Wait()
}

func worker(id int, tasks <-chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for task := range tasks {
        fmt.Printf("Worker %d processing task %d\n", id, task)
        time.Sleep(time.Duration(rand.Intn(500)) * time.Millisecond)
    }
}
```

Преимущества Worker Pool

- Контроль за потреблением ресурсов
- Предсказуемая производительность
- Упрощение управления горутинами
- Балансировка нагрузки
- Высокие накладные расходы

Риски Worker Pool

- Неправильный размер пула
- Блокировки при заполненной очереди
- Простаивающие воркеры
- Сложность масштабирования

Как выбрать размер пула

- 1 Для CPU-bound задач: `runtime.NumCPU()`
- 2 Для I/O-bound задач: экспериментировать
- 3 Динамический пул с автомасштабированием

Динамическое масштабирование

```
var currentWorkers int32 = 5
maxWorkers := 20

go func() {
    for {
        queueSize := len(tasks)
        workers := atomic.LoadInt32(&currentWorkers)

        if queueSize > cap(tasks)/2 && workers < int32(maxWorkers) {
            // Увеличить пул
            atomic.AddInt32(&currentWorkers, 1)
            var wg sync.WaitGroup
            wg.Add(1)
            go worker(tasks, &wg)
        } else if queueSize < cap(tasks)/4 && workers > 5 {
            // Уменьшить пул (сигнал к завершению одного воркера)
            atomic.AddInt32(&currentWorkers, -1)
        }
        time.Sleep(5 * time.Second)
    }
}()
```

Грейсфул шатдаун

```
func main() {  
    tasks := make(chan Task, 100)  
    var wg sync.WaitGroup  
  
    // Запуск воркеров  
    for i := 0; i < 5; i++ {  
        wg.Add(1)  
        go worker(tasks, &wg)  
    }  
  
    // Генератор задач  
    go generateTasks(tasks)  
  
    // Обработка сигналов  
    sigCh := make(chan os.Signal, 1)  
    signal.Notify(sigCh, syscall.SIGINT, syscall.SIGTERM)  
    <-sigCh  
  
    // Завершение  
    close(tasks)  
    wg.Wait()  
}
```

Worker Pool с обработкой результатов

```
func startWorkerPool(numWorkers int) (chan<- Task, <-chan Result) {  
    tasks := make(chan Task, 100)  
    results := make(chan Result, 100)  
    var wg sync.WaitGroup  
  
    for i := 0; i < numWorkers; i++ {  
        wg.Add(1)  
        go func(id int) {  
            defer wg.Done()  
            for task := range tasks {  
                results <- processTask(task)  
            }  
        }(i)  
    }  
  
    go func() {  
        wg.Wait()  
        close(results)  
    }()  
  
    return tasks, results  
}
```

Вопрос

Worker Pool с 10 горутинами и очередь на 100 задач. Что произойдет, если задачи будут приходить быстрее, чем обрабатываться?



Ответ

- Очередь заполнится до 100 задач
- Дальнейшие попытки отправки заблокируются
- Генератор задач остановится

Возможное решение:

- Увеличить размер очереди
- Добавить механизм отбрасывания
- Динамически увеличивать пул



Вопрос

Как правильно завершить Worker Pool, если задачи закончились, но горютины ещё работают?



Ответ

Правильное завершение:

- Закрывать канал задач
- Использовать `sync.WaitGroup` для ожидания
- Дождаться завершения всех воркеров

```
close(tasks) // Сигнал к завершению
wg.Wait()    // Ожидание завершения
close(results)
```





Ваши вопросы?

Сравнение паттернов



4

Сравнительная таблица

Критерий	Fan-out	Fan-in	Worker Pool
Сложность	★★☆☆☆	★★★★☆	★★★★☆
Контроль ресурсов	★☆☆☆☆	★★☆☆☆	★★★★★
Сохранение порядка	Нет	Нет	Нет
Обработка ошибок	Сложная	Средняя	Простая
Использование CPU	Максимальное	Среднее	Контролируемое
Лучший сценарий	Параллельная обработка	Агрегация данных	Управляемая очередь

Когда что использовать?

1

Fan-out

- Обработка большого количества независимых элементов
- Когда порядок не важен
- Вычислительно сложные задачи

2

Fan-in

- Сбор данных из нескольких источников
- Централизованная обработка потоков
- Реализация шаблона Observer

3

Worker Pool

- Ограничение параллелизма
- Системы с ограниченными ресурсами
- Очереди обработки запросов

Комбинирование паттернов

[Источники данных]

↓ (Fan-out)

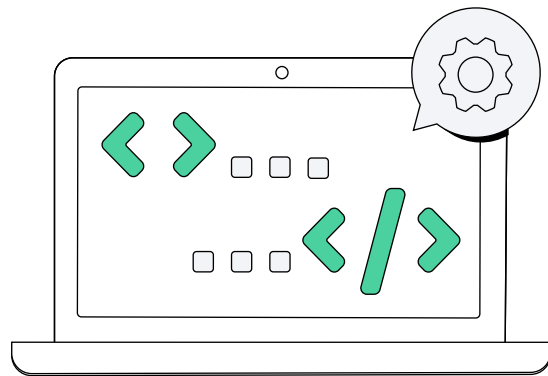
[Обработчики]

↓ (Fan-in)

[Агрегатор]

↓ (Worker Pool)

[Финальная обработка]



Пример конвейера

[Источники данных]

↓ (Fan-out)

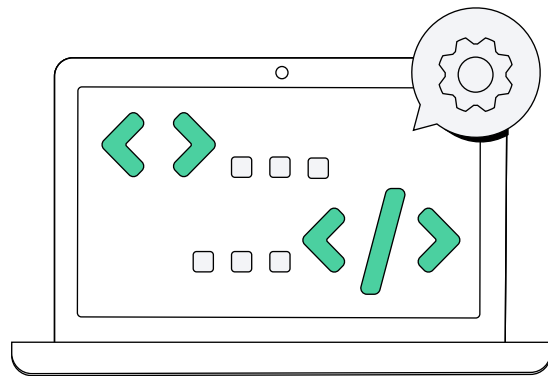
[Обработчики]

↓ (Fan-in)

[Агрегатор]

↓ (Worker Pool)

[Финальная обработка]



Пример конвейера

```
func processingPipeline() {  
    // Этап 1: Fan-out для чтения  
    rawData := readDataSources()  
  
    // Этап 2: Worker Pool для обработки  
    processed := processData(rawData, 10)  
  
    // Этап 3: Fan-in для агрегации  
    result := aggregateResults(processed)  
  
    // Сохранение результатов  
    saveResults(result)  
}
```

Типичные ошибки

- 1 Deadlock из-за незакрытых каналов. Всегда использовать `context/sync.WaitGroup`
- 2 Утечки горутин. Контролировать жизненный цикл
- 3 Блокировки при заполненных каналах. Использовать буферизованные каналы и `select`
- 4 Паники при закрытии каналов. Закрывать только один раз

Лучшие практики

- 1 Всегда ограничивать параллелизм. Семафоры, пулы воркеров
- 2 Использовать context для отмены. Graceful shutdown обязателен
- 3 Закрывать каналы аккуратно. Только отправитель должен закрывать
- 4 Мониторить состояние горутин: runtime.NumGoroutine(), pprof
- 5 Тестировать на deadlock'и: Go race detector, stress-тесты



Ваши вопросы?

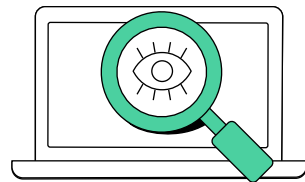
Практика

5



Демонстрация работы

Практика





Ваши вопросы?

Итоги занятия

Сегодня мы

- Разобрали три ключевых паттерна работы с каналами
- Рассмотрели практические кейсы применения
- Сравнили сильные и слабые стороны каждого подхода
- Разобрали типичные ошибки и лучшие практики



Паттерны работы с каналами

Игорь Шамаев
Руководитель направления разработки

